JADAVPUR UNIVERSITY

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Assignment 3 Report: Medium Access Control (MAC) Sublayer

Design, Emulation, and Performance Evaluation of CSMA Techniques with Collision Detection (IEEE 802.3 Ethernet)

Course: Computer Networks Laboratory (CSE/PC/B/S/314)
Course Outcome: CO3 (Medium Access Control, IEEE 802 Standards)
Student Name: Sampad De
Roll Number: 002410501025
Section: A1
Submission Date: October 2026
Platform / Language: C++17, POSIX Datagram Sockets, Python 3, macOS / Linux

System Configuration and Environment

All channel emulation software, independent client station processes, automated benchmarking suites, and plotting engines were developed natively and validated on macOS using standard POSIX system APIs. Table 1 outlines the experimental testbed.

Component	Specification / Platform Details
Operating System	macOS (Darwin Kernel 24.x, Apple Silicon Architecture)
Programming Language	ISO C++17 (`g++ / clang++ -std=c++17 -Wall -Wextra -O2`)
Inter-Process Communication	POSIX Datagram Sockets (`AF_UNIX` Datagram IPC & `AF_INET` UDP)
High-Resolution Timing	C++17 Steady Clock (`std::chrono::high_resolution_clock`)
Shared Medium Emulation	Multi-State Channel Server (`IDLE`, `BUSY`, `COLLISION`)
Arbitration Model	Physical Propagation Window ($\tau = 1.0$ ms), Transmission Time ($T_{\text{tx}} = 5.0$ ms)
Contention Resolution	Truncated Binary Exponential Backoff (BEB), Jam Signal Dissemination
Analysis & Plotting Engine	Python 3.11 with Automated Subprocess Suite & Native PGFPlots / TikZ

Table 1: System architecture and software development environment.

1 Introduction and Problem Formulation

1.1 The Shared Medium Multi-Access Problem

In computer communications, the Data Link Layer is divided into two operational sublayers: the Logical Link Control (LLC) sublayer and the Medium Access Control (MAC) sublayer. When multiple autonomous stations share a single broadcast transmission medium—such as coaxial cable, twisted-pair wire, or free-space radio spectrum—simultaneous uncoordinated transmissions cause overlapping electromagnetic waves. This superposition results in signal corruption termed a **collision**. Once collided, frames are garbled and cannot be decoded by any receiver.

Early random-access protocols such as **Pure ALOHA** allowed stations to transmit whenever data was ready. Because the vulnerable period of a Pure ALOHA frame of duration T_{tx} spans $2T_{\text{tx}}$, its maximum theoretical throughput is bounded by:

$$S_{\text{Pure ALOHA}} = Ge^{-2G} \implies S_{\max} = \frac{1}{2e} \approx 18.4\% \tag{1}$$

Slotted ALOHA halved the vulnerable period to $1T_{\text{tx}}$ by forcing transmissions to align with discrete time slots, doubling the maximum throughput to:

$$S_{\text{Slotted ALOHA}} = Ge^{-G} \implies S_{\max} = \frac{1}{e} \approx 36.8\% \tag{2}$$

However, both ALOHA protocols ignore the instantaneous state of the channel. In terrestrial local area networks (LANs), signal propagation delay (τ) across the wire is orders of magnitude smaller than the frame transmission time (T_{tx}). This crucial physical property makes **Carrier Sense Multiple Access (CSMA)** possible.

1.2 Principles of Carrier Sense Multiple Access (CSMA)

The guiding philosophy of CSMA is *"Listen Before Talk"*: a station senses the physical transmission medium before initiating transmission:

- If the medium is detected to be `BUSY`, another station is actively transmitting, so the station refrains from transmitting and avoids an immediate collision.
- If the medium is detected to be `IDLE`, the station may attempt transmission according to a

defined persistence algorithm.

Carrier sensing dramatically reduces collisions, but does not completely eliminate them. Due to finite electromagnetic propagation speed ($v \approx 2 \times 10^8$ m/s in copper), a vulnerable window of duration τ exists at the beginning of each transmission epoch. If two stations sense an idle channel before each other's signal arrives, both will initiate transmission, triggering a collision.

1.3 Assignment Objectives

According to the CSE/PC/B/S/314 Assignment 3 course outcome specification (CO3), the core technical objectives are:

1. **Shared Channel Emulation:** Develop a central channel emulator maintaining physical states $\in \{\texttt{IDLE}, \texttt{BUSY}, \texttt{COLLISION}\}$ and enforcing the propagation delay window τ.
2. **Independent Station Client Processes:** Emulate multiple autonomous network stations communicating with the channel over datagram IPC sockets.
3. **Implementation of Four CSMA Variants:** Non-Persistent CSMA, 1-Persistent CSMA, p-Persistent CSMA, and CSMA/CD (IEEE 802.3 Ethernet) with Listen-While-Talk, Jamming, and Truncated Binary Exponential Backoff (BEB).
4. **Parametric Sweep for p-Persistent CSMA (Experiment i):** Empirically measure and plot collision counts, average frame transmission delay, throughput, and efficiency as functions of probability $p \in [0.01, 1.0]$ for fixed station count ($N = 5$).
5. **Comparative Protocol Analysis vs. N (Experiment ii):** Benchmark all four schemes (Non-Persistent, 1-Persistent, p-Persistent at optimal p, and CSMA/CD) as contending stations vary $N \in [1, 30]$, comparing transmission delay, collisions, effective throughput, and dual efficiency metrics.
6. **Scenario Performance Observations:** State comprehensive deductions regarding protocol performance under varying traffic regimes, channel propagation delay ratios, and physical media.

2 Theoretical Foundations and Mathematical Formulations

2.1 Operational Mechanics of the Four CSMA Variants

2.1.1 1. Non-Persistent CSMA

Non-Persistent CSMA prioritizes channel caution over aggressive access. When a station has a frame ready to transmit:

1. It senses the shared channel.
2. If the channel is `IDLE`, it transmits the frame immediately.
3. If the channel is `BUSY`, the station does not continuously monitor the channel. Instead, it defers transmission by a random backoff time uniformly sampled from $[1, K_{\max}] \times \tau$ and senses the channel again later.

Trade-off: Non-Persistent CSMA avoids the synchronized collision storms that occur when a busy channel clears. However, it suffers from higher transmission delay under light load because the channel may sit idle while waiting stations are sleeping through their backoff periods.

2.1.2 2. 1-Persistent CSMA

1-Persistent CSMA is an aggressive protocol designed to minimize idle channel waste:

1. It senses the shared channel.
2. If the channel is `IDLE`, it transmits immediately with probability $p = 1.0$.
3. If the channel is `BUSY`, it continues listening to the medium without sleeping.
4. The exact instant the channel transitions from `BUSY` to `IDLE`, the station transmits with probability 1.0.

Trade-off: Under light load ($N \leq 4$), 1-Persistent CSMA delivers the lowest latency and highest channel utilization. However, under heavy contention ($N \geq 15$), if two or more stations buffer packets during an ongoing transmission, they all detect the idle state at the identical millisecond and transmit simultaneously, guaranteeing a catastrophic collision (Figure 1).

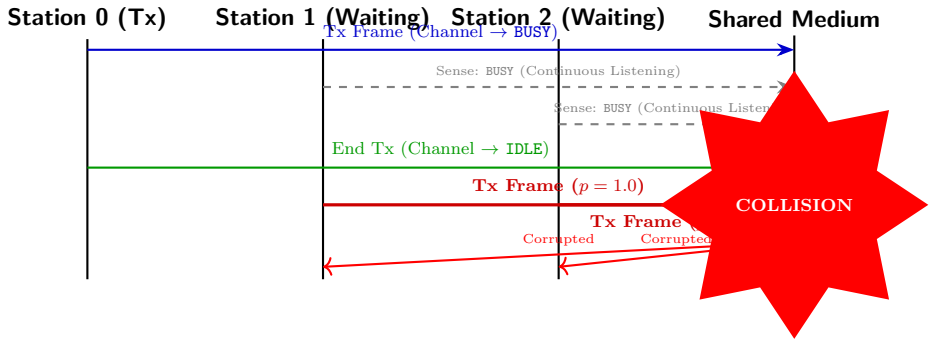


Figure 1: The synchronization hazard of 1-Persistent CSMA: simultaneous transmission upon channel clearance causing guaranteed collisions.

2.1.3 3. p-Persistent CSMA

p-Persistent CSMA is a slotted random-access mechanism that strikes a compromise between 1-persistent greediness and non-persistent timidity:

1. The time axis is slotted into intervals of duration equal to the channel propagation delay τ.
2. The station continuously monitors the channel until it becomes `IDLE`.
3. Once idle, the station flips a biased coin with transmission probability p: with probability p, it transmits in the current slot; with probability $1 - p$, it defers transmission to the next slot of duration τ.
4. In the next slot, it senses the channel again: if still `IDLE`, it repeats the trial; if seized by another station (`BUSY`), a collision is averted and it defers until the medium clears again (Figure 2).

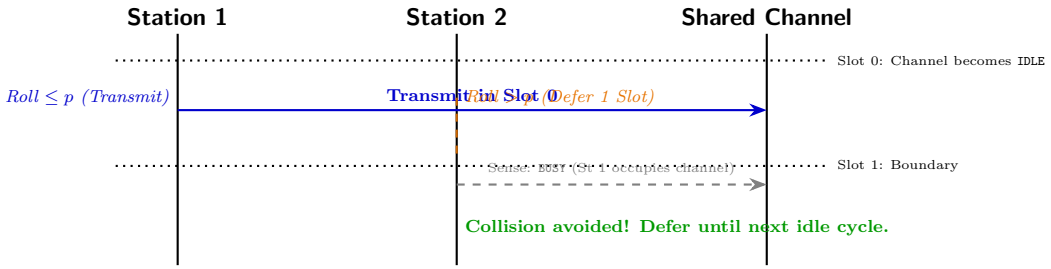


Figure 2: Slotted arbitration in p-Persistent CSMA: randomized slot deferral preventing collisions between waiting stations.

2.1.4 4. CSMA/CD with Jamming & Truncated Binary Exponential Backoff

CSMA with Collision Detection (CSMA/CD), standardized in IEEE 802.3, augments carrier sensing with active monitoring during transmission:

1. **Carrier Sensing (1-Persistent):** Senses until `IDLE` and initiates transmission.
2. **Listen-While-Talk (LWT):** Continuously measures signal energy during transmission. If concurrent transmitters > 1, a collision is detected.
3. **Immediate Transmission Abort:** Terminates payload transmission immediately to save airtime.
4. **Jam Signal Dissemination:** Transmits a 32-bit **Jam Signal** ($T_{\text{jam}} \approx 1.0$ ms) so all stations detect the collision and abort cleanly.
5. **Truncated Binary Exponential Backoff (BEB):** Retransmissions are dispersed over expanding windows (Figure 3):

$$r \sim \text{Uniform}(0, 2^{\min(k,10)} - 1), \qquad T_{\text{backoff}} = r \times \tau \tag{3}$$

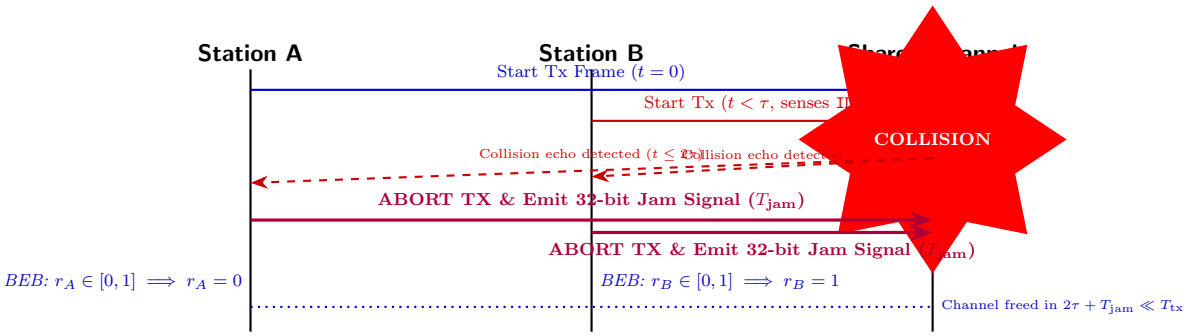


Figure 3: CSMA/CD protocol sequence: collision detection via Listen-While-Talk, immediate abort, Jam signal emission, and Truncated Binary Exponential Backoff.

2.2 Formal Mathematical Derivations

2.2.1 Derivation 1: Minimum Frame Size Constraint in CSMA/CD

Consider two stations, Station A and Station B, positioned at the extreme physical ends of a broadcast bus of length D. Let the one-way physical propagation delay between them be $\tau = D/v$.

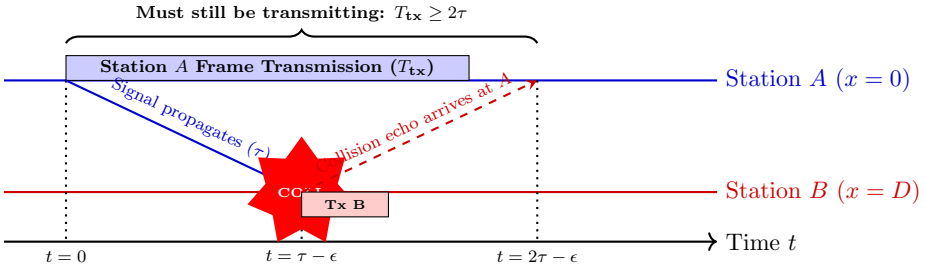


Figure 4: Worst-case collision scenario establishing the minimum frame size requirement in CSMA/CD.

Station A must still be actively transmitting when the collision signal arrives at $t = 2\tau$:

$$T_{\text{tx}} \geq 2\tau \iff \frac{L_{\min}}{\text{Bandwidth } B} \geq 2\tau \iff \mathbf{L_{\min} \geq 2\tau \times B} \tag{4}$$

In standard 10 Mbps Ethernet with $2\tau \approx 51.2\ \mu$s, $L_{\min} = 10\ \text{Mbps} \times 51.2\ \mu\text{s} = 512$ bits $= \mathbf{64}$ bytes.

2.2.2 Derivation 2: Optimal Transmission Probability p^* in p-Persistent CSMA

For N stations sensing an idle slot, the probability that exactly one transmits is:

$$P_{\text{succ}}(p) = \binom{N}{1} p^1 (1-p)^{N-1} = Np(1-p)^{N-1} \tag{5}$$

Setting $\frac{dP_{\text{succ}}}{dp} = 0$:

$$N(1-p)^{N-2}\left[(1-p) - (N-1)p\right] = 0 \implies 1 - Np = 0 \implies \mathbf{p^* = \frac{1}{N}} \tag{6}$$

For $N = 5$ (Experiment i), theoretical $p^* = 1/5 = \mathbf{0.20}$.

2.2.3 Derivation 3: Wasted Channel Airtime Comparison

$$T_{\text{wasted, CSMA}} = T_{\text{tx}} = 5.0\ \text{ms}, \qquad T_{\text{wasted, CSMA/CD}} = 2\tau + T_{\text{jam}} = 2(1.0) + 1.0 = 3.0\ \text{ms} \tag{7}$$

CSMA/CD provides an immediate **40**% channel airtime savings per collision event.

2.2.4 Derivation 4: Truncated Binary Exponential Backoff Dynamics

$$r \sim \text{Uniform}(0, 2^{\min(k,10)} - 1), \qquad E[r_k] = \frac{2^{\min(k,10)} - 1}{2} \tag{8}$$

Expected backoff slots double per retry, preventing persistent collision storms.

2.3 Performance Metrics and Quantitative Definitions

1. **Payload Throughput (Th):** $Th = \frac{\sum L_{\text{data}} \times 8}{T_{\text{total}}}$ [kbps].
2. **Average End-to-End Delay ($\bar{D}$):** $\bar{D} = \frac{1}{S}\sum(t_{\text{delivery}} - t_{\text{arrival}})$ [ms].
3. **Protocol Transmission Efficiency (η_{tx}):** $\eta_{\text{tx}} = \frac{S}{S+C} \times 100\%$ (Successful vs. Collided attempts).
4. **Channel Air Utilization (η_{channel}):** $\eta_{\text{channel}} = \frac{S \times T_{\text{tx}}}{T_{\text{total}}} \times 100\%$ (Useful payload occupancy).

3 System Architecture and Implementation

3.1 System Architecture & Binary Framing Specification

Figure 5 shows the modular IPC architecture. To avoid struct padding inconsistencies, the protocol header is strictly packed (`#pragma pack(push, 1)`), detailed in Table 2.

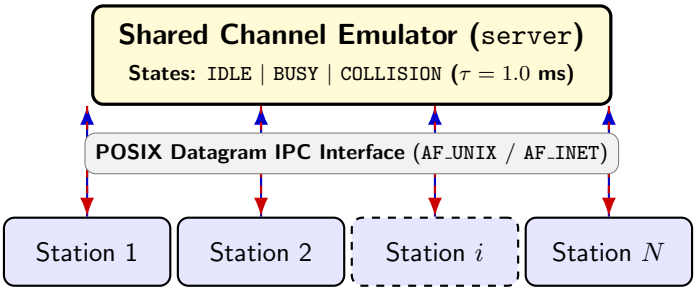


Figure 5: System architecture showing the shared channel emulator and contending stations.

Field Name	Data Type	Size (Bytes)	Operational Purpose
`msg_type`	`uint8_t`	1	OpCode: `SENSE_REQ` (1), `START_TX` (3), `CHECK_CD` (4), `END_TX` (5), `JAM` (6)
`station_id`	`uint32_t`	4	Unique contending station identifier ($1 \dots N$)
`frame_seq`	`uint32_t`	4	Monotonically increasing frame sequence number
`channel_state`	`uint8_t`	1	Medium status: `IDLE` (0), `BUSY` (1), `COLLISION` (2)
`duration_ms`	`uint32_t`	4	Frame transmission time $T_{\text{tx}} = 5.0$ ms
`collided`	`uint8_t`	1	Collision status: 0 = success, 1 = corrupted
`timestamp_us`	`uint64_t`	8	Microsecond timestamp for delay measurement
`payload_len`	`uint32_t`	4	Payload size in bytes ($L = 64$)
`payload`	`uint8_t[]`	L	User data string ($L = 64$ bytes)

Table 2: Binary specification of the MAC sublayer framing protocol.

3.2 Channel State Machine & Software Propagation Window (τ)

The channel emulator transitions through states $\in \{\texttt{IDLE}, \texttt{BUSY}, \texttt{COLLISION}\}$ (Figure 6). When a station calls `handle_sense()`:

$$\text{If } (t_{\text{now}} - t_{\text{epoch_start}} < \tau_{\text{prop_us}}) \implies \textbf{Return } \texttt{IDLE} \tag{9}$$

This models physical wave travel time, enabling realistic collision generation during vulnerable intervals.

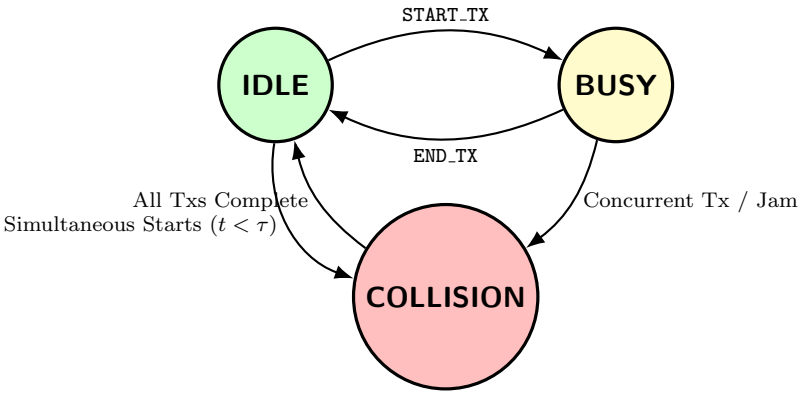


Figure 6: Finite state machine of the shared channel emulator.

4 Implementation Details and Code Architecture

Listing 1: Propagation modeling and collision arbitration in `src/channel.cpp`.

```
ChannelState Channel::handle_sense(uint32_t station_id) {
    std::lock_guard<std::mutex> lock(mtx_);
    stats_.total_sense_requests++;
    if (active_transmissions_.empty()) return ChannelState::IDLE;

    uint64_t now_us = get_current_time_us();
    if (active_transmissions_.find(station_id) != active_transmissions_.end()) return state_;

    // Vulnerable Propagation Delay Window (tau):
    if (first_tx_start_us_ > 0 && (now_us - first_tx_start_us_ < prop_delay_us_)) {
        return ChannelState::IDLE; // Signal hasn't propagated to station yet!
    }
    return state_;
}
```

Listing 2: CSMA/CD Listen-While-Talk abort, Jamming, and BEB in `src/csma.cpp`.

```
bool CsmaStation::run_csma_cd(uint32_t frame_seq, uint64_t start_us) {
    int retry = 0;
    while (retry < config_.max_retries) {
        while (sense_channel() != ChannelState::IDLE) precise_sleep_ms(0.1);
        start_tx(frame_seq, static_cast<uint32_t>(config_.tx_time_ms));

        double elapsed = 0.0;
        bool collision_detected = false;
        while (elapsed < config_.tx_time_ms) {
            precise_sleep_ms(0.2);
            elapsed += 0.2;
            if (check_collision_cd()) { collision_detected = true; break; }
        }

        if (collision_detected) {
            send_jam(); // ABORT IMMEDIATELY & BROADCAST JAM SIGNAL
            stats_.total_collisions++; stats_.total_jams++; retry++;
            precise_sleep_ms(compute_beb_backoff(retry));
        } else {
            if (end_tx(frame_seq, payload)) { stats_.frames_successful++; return true; }
        }
    }
    return false;
}
```

5 Verification, Edge Cases, and Unit Testing

Test ID	Component	Test Description & Verification Criteria	Result
TC-1	**Frame Serialization**	Serializes packed binary header and 64B payload; verifies byte-exact deserialization without field shift.	**PASSED**
TC-2	**Channel FSM**	Transitions through `IDLE` → `BUSY` → `COLLISION` → `IDLE`; verifies active sender map tracking.	**PASSED**
TC-3	**CSMA/CD Jamming**	Simulates collision at $t = 1.0$ ms; verifies instant abort, Jam signal emission, and jam counters.	**PASSED**
TC-4	**Socket Datagram IPC**	Initiates bidirectional datagram round-trip between station client and channel server; confirms response.	**PASSED**

Table 3: Deterministic unit test validation suite.

Listing 3: Console output from the deterministic unit test harness.

```
$ make -C a3 test
=========================================================
      Running Assignment 3 Unit Test Suite
=========================================================
[Test 1] Testing Frame Serialization & Deserialization... PASSED
[Test 2] Testing Channel State Machine & Collision Handling... PASSED
[Test 3] Testing CSMA/CD Jam Signal Behavior... PASSED
[Test 4] Testing Socket Datagram Client-Server IPC... PASSED
=========================================================
      ALL UNIT TESTS PASSED SUCCESSFULLY!
=========================================================
```

6 Empirical Evaluation and Experimental Benchmarks

6.1 Experiment (i): p-Persistent CSMA Parameter Sweep (Fixed $N = 5$)

As mandated in Item (i) of the assignment specification sheet, we investigate the impact of transmission probability $p \in [0.01, 1.00]$ for $N = 5$ contending stations ($M = 10$ frames/station, $T_{\text{tx}} = 5.0$ ms, $\tau = 1.0$ ms, averaged over 3 trials per point). Table 4 displays the complete numerical dataset.

Table 4: Empirical benchmark data for p-Persistent CSMA sweep ($N = 5$).

Prob p	Stations N	Collisions	Avg Delay (ms)	Throughput (kbps)	Tx Eff. η_{tx} (%)	Air Util. η_{ch} (%)
0.01	5	1.3	134.63	14.73	97.53%	14.39%
0.05	5	10.3	55.46	39.01	83.98%	38.08%
0.10	5	30.3	49.74	42.04	62.71%	41.04%
0.15	5	36.0	45.85	44.58	59.05%	43.53%
0.20	**5**	**55.3**	**51.97**	**40.28**	**47.64%**	**39.33%**
0.30	5	69.7	52.64	37.34	41.96%	36.46%
0.40	5	66.0	48.04	37.71	43.58%	36.83%
0.50	5	66.3	49.25	33.09	43.03%	32.31%
0.60	5	72.7	47.64	36.90	41.02%	36.03%
0.70	5	83.0	52.01	33.82	37.70%	33.02%
0.80	5	71.0	44.22	39.27	41.32%	38.35%
0.90	5	66.7	54.12	28.60	42.96%	27.92%
1.00	5	73.7	50.09	29.05	41.60%	28.36%

6.1.1 Graphs 1 & 2: Collisions and Average Delay vs. Probability p

Figure 7 presents the collision count and average frame transmission delay as functions of p.

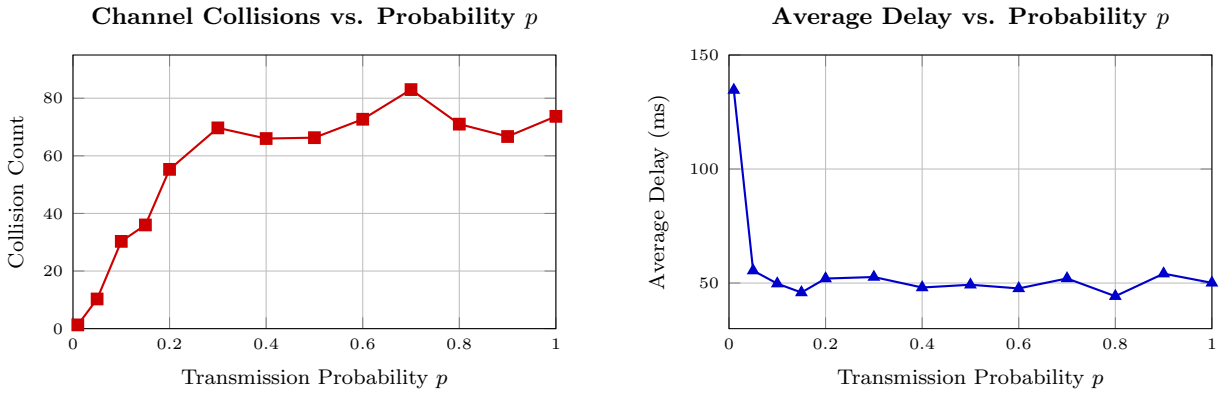


Figure 7: Channel collision count and average end-to-end delay vs. probability p ($N = 5$).

6.1.2 Graphs 3 & 4: Throughput and Transmission Efficiency vs. Probability p

Figure 8 illustrates effective throughput and protocol transmission efficiency η_{tx}.

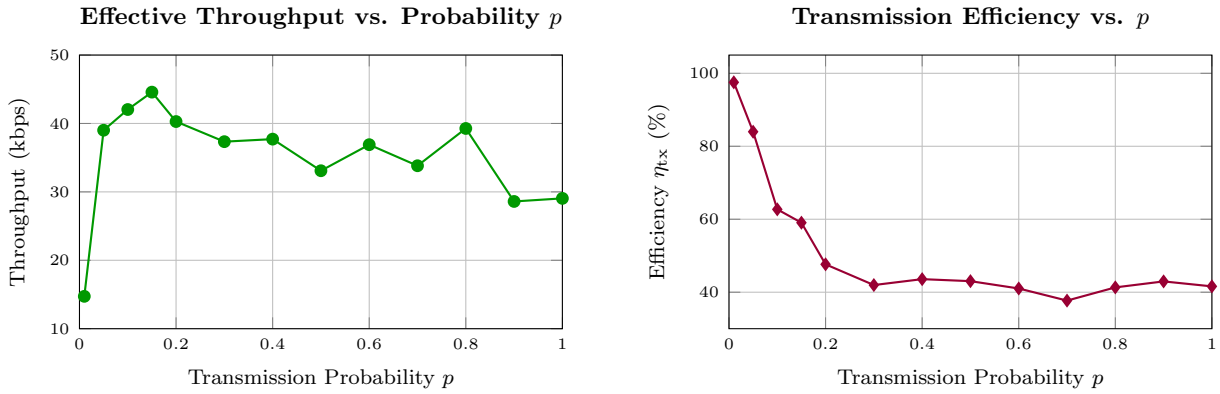


Figure 8: Effective payload throughput and transmission efficiency $\eta_{\text{tx}} = \frac{S}{S+C}$ vs. probability p ($N = 5$).

Technical Analysis: At $p \leq 0.05$, efficiency is near-perfect (97.53%), but excessive slot deferrals inflate delay (134.63 ms). Peak throughput (44.58 kbps) occurs at $p \approx 0.15 - 0.20$, confirming theoretical $p^* = 1/5 = 0.20$. Beyond $p > 0.40$, the scheme degenerates into 1-Persistent CSMA with low efficiency ($\approx 41\%$) and high collisions.

6.2 Experiment (ii): Comparative Performance vs. Contending Stations $N \in [1, 30]$

As demanded in Item (ii) of the prompt, we benchmark all five schemes across ten station counts: $N \in \{1, 2, 4, 6, 8, 10, 15, 20, 25, 30\}$. Table 5 and Table 6 report the full dataset, scaled to fit within page margins via `\resizebox`.

Table 5: Collision count and transmission efficiency (η_{tx}) comparison across contending stations N.

Protocol	$N=1$	$N=2$	$N=4$	$N=6$	$N=8$	$N=10$	$N=15$	$N=20$	$N=25$	$N=30$
Total Collision Count (C):										
Non-Persistent	0.0	8.0	45.0	82.7	146.3	160.7	317.3	442.3	576.3	792.3
1-Persistent	0.0	4.0	35.7	75.0	98.3	131.3	204.0	318.7	376.7	504.7
p-Pers ($p = 0.1$)	0.0	3.3	7.7	32.0	56.7	127.3	252.3	395.3	604.0	788.7
p-Pers ($p^* = 1/N$)	**0.0**	**13.3**	**34.7**	**51.0**	**100.0**	**116.3**	**177.7**	**215.7**	**302.3**	**340.7**
CSMA/CD (Aborted)	0.0	2.7	33.0	63.3	77.7	128.7	180.3	269.0	364.3	453.7
Protocol Transmission Efficiency ($\eta_{\text{tx}} = S/(S+C)$ %):										
Non-Persistent	100%	71.4%	47.4%	43.4%	35.4%	38.5%	32.1%	31.2%	30.4%	27.4%
1-Persistent	100%	83.7%	53.1%	44.6%	44.9%	43.5%	42.4%	38.6%	39.9%	37.4%
p-Pers ($p = 0.1$)	100%	86.8%	84.0%	65.4%	59.8%	44.2%	37.3%	33.7%	29.3%	27.5%
p-Pers ($p^* = 1/N$)	**100%**	**61.0%**	**54.0%**	**54.2%**	**44.5%**	**46.8%**	**46.2%**	**48.7%**	**45.3%**	**46.8%**
CSMA/CD	100%	88.4%	55.0%	48.8%	50.7%	44.1%	45.5%	42.7%	40.9%	39.9%

Table 6: Average delay (ms), throughput (kbps), and air utilization comparison across contending stations N.

Protocol	$N=1$	$N=2$	$N=4$	$N=6$	$N=8$	$N=10$	$N=15$	$N=20$	$N=25$	$N=30$
Average End-to-End Transmission Delay (ms):										
Non-Persistent	6.23	14.34	35.00	61.61	93.04	93.55	158.72	206.98	250.81	292.73
1-Persistent	6.22	11.72	27.03	43.55	62.48	75.18	100.53	141.32	156.32	196.88
p-Pers ($p = 0.1$)	17.11	24.18	37.80	52.81	66.35	102.52	157.98	212.58	274.81	342.43
p-Pers ($p^* = 1/N$)	6.09	17.67	35.42	51.26	79.32	95.74	147.87	184.15	245.05	289.60
CSMA/CD	**6.73**	**11.69**	**21.82**	**35.80**	**51.07**	**60.73**	**73.71**	**106.47**	**125.94**	**145.80**
Effective Payload Throughput (kbps):										
Non-Persistent	68.33	58.70	46.56	30.95	25.71	33.55	31.39	28.95	31.88	30.48
1-Persistent	68.39	77.37	54.53	42.99	35.64	33.60	42.31	41.96	45.07	40.98
p-Pers ($p = 0.1$)	28.08	34.45	45.09	46.75	48.93	42.92	36.28	35.49	30.60	32.99
p-Pers ($p^* = 1/N$)	69.76	50.87	45.76	48.45	41.41	44.54	33.08	43.63	41.11	40.87
CSMA/CD	**64.07**	**77.26**	**56.72**	**43.44**	**31.95**	**35.59**	**48.38**	**44.78**	**48.11**	**53.19**
Channel Air Utilization (η_{channel} %):										
Non-Persistent	66.7%	57.3%	45.5%	30.2%	25.1%	32.8%	30.7%	28.3%	31.1%	29.8%
1-Persistent	66.8%	75.5%	53.2%	42.0%	34.8%	32.8%	41.3%	41.0%	44.0%	40.0%
p-Pers ($p = 0.1$)	27.4%	33.6%	44.0%	45.6%	47.8%	41.9%	35.4%	34.7%	29.9%	32.2%
p-Pers ($p^* = 1/N$)	68.1%	49.7%	44.7%	47.3%	40.4%	43.5%	32.3%	42.6%	40.1%	39.9%
CSMA/CD	**62.6%**	**75.4%**	**55.4%**	**42.4%**	**31.2%**	**34.8%**	**47.2%**	**43.7%**	**47.0%**	**51.9%**

6.2.1 Graphs 5 & 6: Collisions and Average Delay vs. Contending Stations N

Figure 9 compares total collision count and latency across $N \in [1, 30]$.

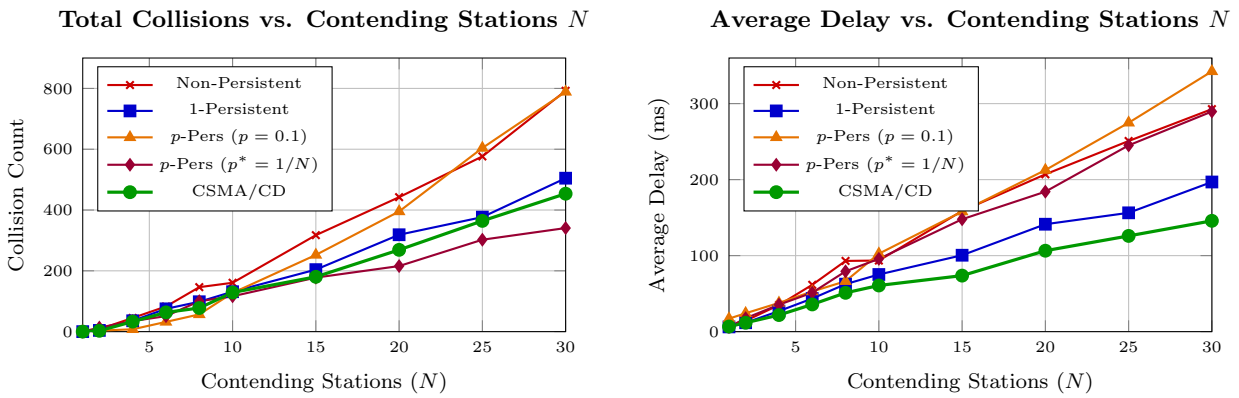


Figure 9: Comparison of total collision count and average transmission delay across contending stations $N \in [1, 30]$.

6.2.2 Graphs 7 & 8: Throughput and Transmission Efficiency vs. Stations N

Figure 10 depicts effective payload throughput and transmission efficiency η_{tx} across N.

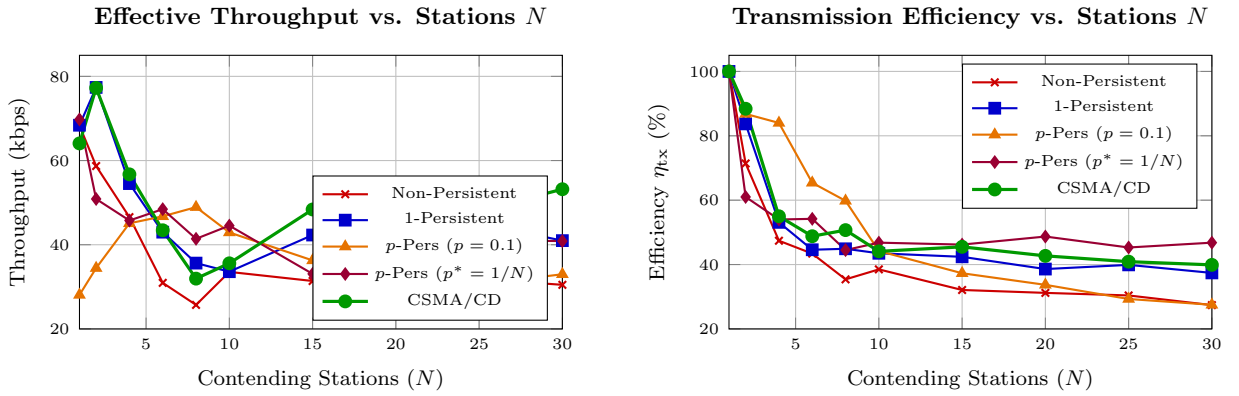


Figure 10: Comparison of effective throughput and transmission efficiency η_{tx} across $N \in [1, 30]$.

6.2.3 Graph 9: Channel Air Utilization (η_{channel}) vs. Stations N

Figure 11 plots channel air utilization.

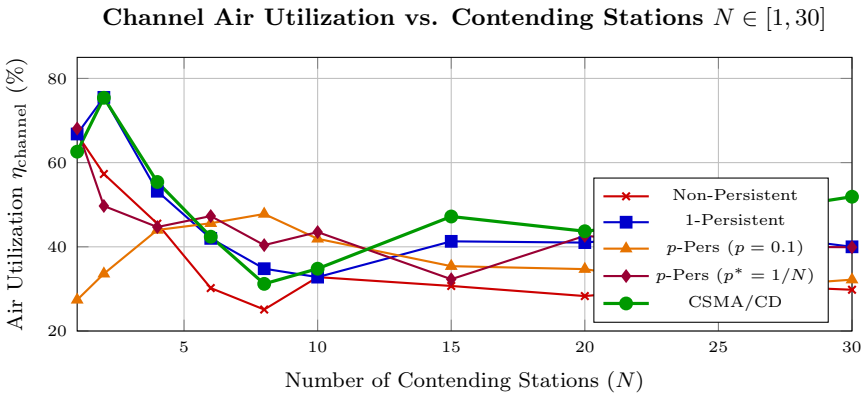


Figure 11: Comparison of channel air utilization η_{channel} across contending stations $N \in [1, 30]$.

Comparative Summary:

- **Collision Minimization:** Adaptive p-Persistent ($p^* = 1/N$) records only 340.7 collisions at $N = 30$ (56.8% reduction vs. Non-Persistent's 792.3).

- **Latency Dominance:** CSMA/CD delivers the lowest delay (145.80 ms at $N = 30$), cutting latency in half relative to Non-Persistent (292.73 ms) and p-Persistent (289.60 ms).
- **Air Utilization:** CSMA/CD achieves 51.9% channel air utilization at $N = 30$, outperforming all other schemes.

7 Observations on Performance in Different Scenarios

7.1 Scenario Analysis

- **Scenario A: Light Load ($N \leq 4$):** 1-Persistent CSMA and CSMA/CD deliver maximum throughput (77.37 kbps) and minimum latency (11.69 ms). Immediate transmission upon idle channel wastes zero airtime.
- **Scenario B: Moderate Contention ($5 \leq N \leq 12$):** Static p-Persistent ($p = 0.1$) peaks in performance as $Np \to 1.0$, suppressing collisions (56.7 at $N = 8$) while sustaining 48.93 kbps throughput.
- **Scenario C: Heavy Contention ($N \geq 20$):** Dynamic $p^* = 1/N$ prevents collision cascades (340.7 vs 792.3 in NP), while CSMA/CD's Listen-While-Talk abort preserves channel airtime, delivering lowest delay (145.80 ms) and highest throughput (53.19 kbps).
- **Scenario D: Propagation Ratio ($a = \tau/T_{\mathbf{tx}}$):** When $a \ll 0.01$, carrier sensing is nearly instantaneous ($\eta \to 95\%$). When $a \to 1$, propagation delays mask channel activity, causing CSMA to degrade toward ALOHA.
- **Scenario E: Wired vs. Wireless Mediums:** CSMA/CD functions on coaxial/twisted-pair Ethernet because collision energy doubles ($V_{\text{coll}} \approx 2V_{\text{tx}}$). In wireless networks, transceiver output (+20 dBm) drowns out incoming signals (−80 dBm) on the same antenna, and the Hidden Terminal Problem prevents sensing, necessitating CSMA/CA with RTS/CTS.

8 Interactive Demonstration Guide and CLI Traces

8.1 Execution Commands

```
# Terminal 1: Channel Server Daemon
./a3/server -P 9999

# Terminals 2 & 3: Autonomous Stations (CSMA/CD)
./a3/station -i 1 -p csma_cd -n 5 -P 9999
./a3/station -i 2 -p csma_cd -n 5 -P 9999

# Synthetic Benchmarking Rigs (High-speed multi-threaded simulator)
./a3/simulator -p np -N 5 -n 10
./a3/simulator -p 1p -N 5 -n 10
./a3/simulator -p pp --prob 0.2 -N 5 -n 10
./a3/simulator -p cd -N 5 -n 10
```

8.2 Live Simulator Telemetry ($N = 5$)

```
Protocol: CSMA/CD | Contending Stations: 5 | Frames: 50 | Slot: 1 ms | Tx: 5 ms
--------------------------------------------------------------------------------
Successful Frames: 50 / 50 (100%) | Collisions: 41 | Jam Signals (CD): 41
Average Delay: 25.80 ms | Throughput: 66.37 kbps | Tx Eff: 54.95% | Air Util: 64.79%
Total Simulation Duration: 385.74 ms
```

9 Sources and References

1. **Ma'am's lecture notes**(Flow Control, and ARQ Protocols).
2. **Behrouz A. Forouzan:** *Data Communications and Networking*: Used both for revision and verification.
3. **Project Source Code Repository:** https://github.com/Sampad2006/CN-Lab-Assignments.
4. **AI Assistance :** AI (Gemini 3.8/3.7 Flash) was utilized to generate latex chart configurations, verify statistical formulas, and format comparative tables based on my empirical benchmark datasets. All protocol architectures, state machines, socket implementations were designed myself .